FIT2101: Software Engineering Process and Management

Application Name: *"Tuesday"*

Summary

Area of Focus	Languages/frameworks considered	Recommended language/framework	Explanation
Programming Language	C++ Ruby JavaScript Swift	JavaScript	Most flexible and most familiar programming language for development team
Platform / Architecture	iOS/Android Native Application Web Application	Web Application	Most suitable and flexible option and is primarily coded in JavaScript
Front-end framework	Native JavaScript with Material Design Lite Native JavaScript with Storybook React with Material UI	React with Material UI	Whilst the development team is not as familiar as other options, React / Material UI provides comprehensive features as well as stylish pre-defined UI components
Back-end software	Local Storage (browser-based) Local Storage (JSON file) MongoDB MySQL	Local Storage (browser-based)	Current use of a single computer for running the application means using browser-based Local Storage is the simplest option to implement whilst still covering all requirements

Table 1. Overall summary of analysis

Programming Language

The following terms of reference are considered for Programming Languages:

- Experience with the programming language by the development team
- Ease of development, especially with unfamiliar languages
- What type of platform(s) the language supports
- Where applicable, specific benefits of the programming language over other languages

C++:
C++ is best designed for creating native applications and can also be used to develop cross-platform mobile applications on both iOS and Android [2]. C++ is considered a difficult programming language, especially for those who have little experience with memory management [1]. However, the benefit of C++, especially compared to other languages, is the ability to create highly efficient and performant programs thanks to the aforementioned memory management capabilities.

Our development team has minimal experience with C++. However, if the platform decided on is a native application, then C++ would be a good programming language to use.

Ruby:
Ruby is a general-purpose, high-level programming language [3]. It can be used to create both cross-platform native applications via extensions such as RubyMotion [4] as well as web applications via extensions such as Ruby on Rails [5]. Ruby / Ruby on Rails is especially powerful for creating performant web applications and is used by many major online platforms, including GitHub and Zendesk [5].

Our development team has minimal experience with Ruby. However, it is a widely used language that is considerably easier to learn than C++ and so is a viable option for this project.

JavaScript:
JavaScript is a very common go-to language when developing web applications. Web applications can be created from base JavaScript using no frameworks via templates such as Material Design Lite [6] or additional functionality can be added using frameworks such as React [7]. JavaScript is a fairly easy language to learn and is often compared in ease of learning to languages such as Python.

Our development team has extensive experience with JavaScript and have completed multiple projects in prior subjects.

Swift:
Swift is Apple's general-purpose programming language for creating applications that target the iOS architecture [8]. It is marketed at being easy to learn and is purpose-built for creating iOS applications [8]. Swift, unlike other programming languages mentioned above, is designed with safety in mind and has multiple built-in error-checking capabilities.

Our development team has some experience with Swift, with one member having completed projects previously. However, Swift has comprehensive documentation and has a vast amount of teaching resources.

Recommendations:
The programming language recommended is dependent on the type of platform that is being targeted by the project. Thus, whilst the recommended platform is a Web Application that uses JavaScript (as discussed in the section below), there are other recommendations if the recommend platform changes.

Programming Language	Platform(s) supported	Ease of programming	Main benefits	Team Experience
C++	Native and mobile (iOS and Android) applications	Complex compared to other languages	High performance compared to previous languages	Low
Ruby	Native and web applications	Beginner friendly	Multiple frameworks exist to support different architectures	Low
JavaScript	Web applications	Beginner friendly	Multiple frameworks exist to support different architectures	High
Swift	iOS applications	Beginner friendly	Purpose designed for creating iOS applications	Moderate

Table 2. Summary of programming languages considered

If the platform is a web application, then JavaScript is the recommended language given its flexibility with frameworks and familiarity with the development team. This will allow the project to be completed without requiring the development team to learn an entirely new language, helping to keep the project on schedule and reduce the risk of delays.

If the desired platform is a native application that targets Windows, the recommend language is Ruby. Ruby and C++ are the main programming languages considered and given the lack of knowledge of both languages by the development team, using a higher-level language bsuch as Ruby is preferred such that the ease of development is greatest. Additionally, whilst C++ does offer greater performance, Ruby can provide performance that is more than satisfactory.

Platform / Architecture

The following terms of reference are considered:

- Experience with the platform by the development team
- Flexibility of the platform to support further features, for example client-server architecture[1]
- Ability to meet all requirements currently specified by the client
- Cross-platform capabilities
- Usability / cleanliness of user interface considering the amount of information displayed per page

iOS / Android Application:
Using languages mentioned above such as Swift (for iOS development) or C++ (for iOS and Android development), an application can be created on these mobile devices. Both programming languages support server architecture and should have the capability to meet all the requirements specified by the client.

An application built in Swift would be limited to iOS devices only, whereas an application built in C++ can be created to be cross-platform.

Furthermore, the main limitation of a mobile application is the amount of information that needs to be displayed on each page. Given the small screen on a mobile device, it is likely that each page will be cluttered or will be unable to display as much information as a non-mobile alternative.

Additionally, as mentioned above, the development team have minimal experience in mobile-targeted development.

Native Application:
Using C++ or Ruby, a native application would primarily target Windows or possibly iOS desktops and laptops. Both languages support server-client architecture and have the capabilities required to meet the client requirements.

A native application would be limited to the system it is developed for, i.e. the application would be Windows or Mac only without additional effort to create a separate version for other operating systems.

However, as a native application would be run on a desktop/laptop, the screen size will be considerably larger than a mobile device and thus more information will be able to be displayed on the screen / information will be less cluttered and more readable.

Similar to iOS/Android development, the development team have minimal experience in native application development.

Web Application:
Using JavaScript or Ruby on Rails, a web application would target all major browsers (Chrome, Firefox, etc.) Both languages support server-client architecture and, with various modules (see below section), can support all the functionality required.

[1] Whilst client-server architecture is not required as part of the client specifications provided, the original project brief states that “future” versions will be on client-server architecture and so this is factored into this analysis.

The benefit of a web application is that it can support most architectures (Windows, Mac, mobile devices) provided the application is coded to support different screen sizes. Additionally, assuming the primary architecture is for a desktop, this will allow all information to be displayed on a page without it being cluttered.

On top of this, the development team have fairly extensive experience, specifically in JavaScript.

Recommendations:
The desired platform / architecture will heavily influence which programming language is used.

Platform / Architecture	Support for additional functionality	Cross platform capabilities	Limitations	Team Experience
iOS (Swift, C++) or Android (C++)	Yes	C++: Yes, with additional coding to target different platform	Mobile screens may be insufficient given amount of information required	Low
Native Application (C++, Ruby)	Yes	Yes, with extensive additional coding to target different platform	Cross-platform requires extensive additional coding	Low
Web Application (JavaScript, Ruby)	Yes	Yes, with additional coding to ensure UI works on mobile	Certain browsers have limitations, though the desktop used can be configured with a specific browser in mind	High

Table 3. Summary of platforms/architectures considered

Having examined the main platforms that could be used, a web application is recommended for multiple reasons:

- It can support multiple platforms, including desktops and mobile devices, with only changes to the UI required
- The development team has considerable experience with web development, specifically with JavaScript.

As such, it is recommended that the project be primarily written in JavaScript with a web application being the platform of choice.

Front-End Framework

Operating under the assumption that the project will create a web application using JavaScript, the specific framework for user interface is now required to be decided upon.

The following terms of reference are considered:

- Familiarity with the framework by the development team
- Usability / customizability of the framework to support complex features
- The ability / difficulty required to create a user-friendly/visually attractive user interface
- The readability of the code of each framework

Native JavaScript via Material Design Lite:
Material Design Lite is a component library that allows developers to create a user interface with only default JavaScript, HTML and CSS. The component library is comprehensive and includes elements from entire layouts to specific types of toggles. It is customisable by modifying the CSS and HTML to add additional MDL components.

There are two main downsides of Material Design Lite:

- Adding interactions with components can be unintuitive, especially when handling multi-component interactions [9]
- Readability of the code is poor, with each component having large class names (e.g. “mdl-button mdl-js-button mdl-button--raised mdl-js-ripple-effect mdl-button—accent” [9]

However, the development team have used Material Design Lite quite extensively in previous projects.

A spike was created using Material Design Lite and is available in the Sandbox folder in the team’s repository.

Native JavaScript with own components via Storybook:
Storybook is a UI component development and testing platform [10]. It allows developers to easily check how individual UI component’s display and rapidly prototype changes to each individual component.

This would be used to create unique components specifically for this project which will provide a greater deal of customisability for the user interface.

However, Storybook does not assist with the actual code behind each component, and components would need to be created from scratch. This is a challenging process, especially for development teams that do not have much front-end/user interface experience, which includes our development team.

Additionally, given the relatively short timeframe that the entire project is being completed with, the time spent learning and creating unique components may risk the project not being completed in time.

React via Node.JS and Material UI:
React is a Node.JS library designed to create responsive and component-based user interfaces. It supports developers in creating user interfaces by being rapidly testable with interaction between components in mind. [11]

Additionally, Material UI is a React-based component library that is built on top of Material Design Lite with the purpose of allowing developers to take advantage of React's interaction handling whilst being able to use the designs available in Material Design Lite. [12]

Material UI is also one of the biggest and most popular React component libraries [13] and offers a free version that includes all user interface features that should be required to complete the project.

The use of React and Material UI allows developers to create good-looking UIs whilst also having readable and modular code.

The development team have minimal experience with React and Material UI, however extensive documentation and learning resources exist. Additionally, a spike was created in React with Material UI which determined the framework(s) are feasible to be learnt in a short timeframe. The spike can be viewed in the spike/react-testing branch in the team repository.

Recommendations:

Front-end Platform	**Usability / flexibility**	**Ease of creating visually pleasing components**	**Readability of code**	**Team Experience**
Native JavaScript / Material Design Lite	Usable, but interactions between components can be unintuitive or convoluted	Easy; elements are pre-designed and customisable	Poor	High
Native JavaScript / Storybook	Highly usable for testing components, but requires the developer to create the components	Complex; developers must create their own components that can be tested within Storybook	N/A – components are built by developers in JavaScript and is developer-dependent	Low
React / Material UI	Highly usable and designed for interactivity	Easy; elements are pre-designed and customisable	Moderate, components are modularised though can become complex	Low

Table 4. Summary of front-end platforms considered

Whilst the development team has more experience with base JavaScript and Material Design Lite, React / Material UI provide extensive interaction capacity whilst also being more readable than Material Design Lite.

Given the spike confirming the viability of using React/Material AI, as well as the opportunity afforded by this project to learn a new framework, the development team recommends the use of React/Material AI as it will both provide a more readable source code whilst also providing more functionality than other options.

Back-End Software

The last overall technological decision is the back-end software/mechanism to use.

The following terms of reference are used:

- Familiarity with the back-end software by the development team
- Whether the software can store all the information required according to the specifications
- How simple the software is to use given the specific functionality required
- Any additional considerations that would be required (e.g. hosting a database server either on the local computer or on a server).

Local Storage (Browser-based):
Modern browsers allow data to be stored directly in the browser without the need for any additional servers being run. Data, in the form of JSON, can be stored and retrieved from the browser easily and rapidly via JavaScript.

A limitation of Local Storage is that the storage is browser-dependent and switching browsers will result in data being lost. This will need to be a consideration if the program architecture becomes client-server, however, would be sufficient for the current requirements.

The development team has used Local Storage extensively in prior projects.

Local Storage (JSON Files):
Another local storage option is having stored JSON files on the computer that can be read and written to. This allows data to be accessed across multiple browsers with any arbitrary data stored.

However, Node.JS (and by extension React) has issues with files that are constantly updated (when files are opened, wrote to, then closed) and does not support multiple simultaneous changes to the file by different clients. As such, using JSON files can have issues when multiple connections are made, making it unsuitable for any future modifications to make the architecture client-server.

The development team has some experience with using local JSON files in prior projects.

MongoDB:
MongoDB is a document-driven, non-relational SQL database. It allows for arbitrary data to be stored without having strict database configuration requirements [14] and would be able to store all required data. Using a database such as MongoDB would allow for multiple simultaneous connections to the information as well as supporting any future upgrades of the architecture to be client-server.

However, using a database such as MongoDB requires a server to be hosted (either on the current machine or on a remote server), which adds additional installation and maintenance of the software.

Learning MongoDB is also more complex than previous options.

The development team has limited experience with using MongoDB.

MySQL:
MySQL is a relational SQL database [15]. It requires strict database configuration and data entry. It has similar benefits as MongoDB with the additional benefit that the strict language requirements and relational configuration increases performance when configured correctly.

MySQL has the same requirements as MongoDB with regards to a server being hosted. MySQL is also considered a complex language to learn.

The development team has some experience with MySQL.

Recommendations:

Back-end Platform	Data storage method	Simplicity	Additional considerations / limitations	Team Experience
Local Storage (Browser-based)	JSON in the browser	Easy to use	Data storage only works in the specific browser	High
Local Storage (JSON files)	JSON on the computer	Easy to use	Data storage has read/write limitations and is device-specific	Moderate
MongoDB	Arbitrary data storage in database	Moderately complex	Additional software (database) required to be installed	Low
MySQL	Specific data storage in defined tables	Advanced	Additional software (database) required to be installed	Moderate

Table 5. Summary of back-end end platforms considered

Given the relatively short time-frame that the project is being completed in, and the current requirement that the software be client-side only, the usage of a database such as MongoDB and MySQL introduces additional complexity that does not appear warranted. Additionally, testing and automation becomes more challenging when multiple systems are required.

As such, the development team recommends the usage of Local Storage in the browser as not only does the team have extensive experience using it, but the limitation of being locked to a specific browser is not a major setback as the software will be used only on a specific computer and hence the browser can be restricted to a single browser. This also prevents the issues from local JSON file storage from impacting performance of the program.

References

[1]E. Schaffer, "Is C++ still a good language to learn for 2022?", Educative, 2022. [Online]. Available: https://www.educative.io/blog/learn-cpp-for-2022.

[2]"Mobile development with C++ documentation", Docs.microsoft.com. [Online]. Available: https://docs.microsoft.com/en-us/cpp/cross-platform/?view=msvc-170.

[3]"Ruby (programming language) - Wikipedia", En.wikipedia.org. [Online]. Available: https://en.wikipedia.org/wiki/Ruby_(programming_language).

[4]"Write cross-platform native apps in Ruby | RubyMotion", Rubymotion.com. [Online]. Available: http://www.rubymotion.com/.

[5]"Ruby on Rails", Ruby on Rails. [Online]. Available: https://rubyonrails.org/.

[6]"Material Design Lite", Getmdl.io. [Online]. Available: https://getmdl.io/.

[7]"React – A JavaScript library for building user interfaces", Reactjs.org. [Online]. Available: https://reactjs.org/.

[8]"Swift - Apple Developer", Developer.apple.com. [Online]. Available: https://developer.apple.com/swift/.

[9]"Material Design Lite", Getmdl.io. [Online]. Available: https://getmdl.io/started/index.html#dynamic.

[10]"Storybook: Build component drivern UIs faster", Storybook.js.org. [Online]. Available: https://storybook.js.org/.

[11]"React – A JavaScript library for building user interfaces", Reactjs.org. [Online]. Available: https://reactjs.org/.

[12]"MUI: The React component library you always wanted", Mui.com. [Online]. Available: https://mui.com/.

[13]"23 Best React Component Libraries for 2022 | Technostacks", Technostacks Infotech, 2022. [Online]. Available: https://technostacks.com/blog/react-component-libraries/.

[14]"MongoDB: The Developer Data Platform | MongoDB", MongoDB. [Online]. Available: https://www.mongodb.com/.

[15]"MySQL :: MySQL 8.0 Reference Manual :: 1.2.1 What is MySQL?", Dev.mysql.com. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/what-is-mysql.html.